

Documentation interne

Nom du projet

Groupe:

L3Wx

Encadrant:

Prénom NOM

Auteurs:

NOM Prenom, NOM2 Prenom2

Version du document:

1.0

Résumé

Ce document constitue la documentation technique interne du projet. Il décrit l'architecture du système, les choix techniques, les mécanismes d'intégration des composants, ainsi que les aspects de sécurité et de maintenance.

Table des matières

1	Introduction	3
1.1	Objectifs fonctionnels et techniques	3
1.1.1	Objectifs fonctionnels	3
1.1.2	Objectifs techniques	3
1.2	Architecture du projet	3
2	Description des composants applicatifs	3
2.1	Composant 1 : [Nom]	3
2.2	Composant 2 : [Nom]	4
3	Architecture back-end	4
3.1	Structure du projet	4
3.2	Modèles de données	4
3.3	API REST	4
4	Intégration des services externes	5
4.1	Service de messagerie	5
4.2	Intégration d'un service tiers	5
5	Traitement des données	5
5.1	Formats pris en charge	5
5.2	Pipeline de traitement	5
6	Fonctionnalités additionnelles	6
6.1	Historique et journalisation	6
6.2	Accessibilité	6
7	Sécurité de l'application	6
7.1	Authentification et gestion des sessions	6
7.2	Contrôle d'accès	6
7.3	Sécurité des données	6
7.4	Tests de sécurité	6
8	Conclusion	6

1 Introduction

1.1 Objectifs fonctionnels et techniques

[Décrire ce que le système doit faire du point de vue fonctionnel et technique.]

1.1.1 Objectifs fonctionnels

L'application vise à fournir les fonctionnalités suivantes :

1. Fonctionnalité principale :

option sous-objectif 1

option sous-objectif 2

2. Fonctionnalité secondaire :

option sous-objectif 1

option sous-objectif 2

3. Gestion des comptes :

- ▶ Inscription, connexion, réinitialisation de mot de passe.
- ▶ Distinction entre utilisateurs standard et administrateurs.

1.1.2 Objectifs techniques

- ▶ **Performance** : temps de réponse inférieur à [N] secondes.
- ▶ **Scalabilité** : architecture permettant de gérer la montée en charge.
- ▶ **Sécurité** : authentification sécurisée, chiffrement des données.
- ▶ **Maintenabilité** : code modulaire, commenté et versionné.

1.2 Architecture du projet

[Décrire l'architecture globale du projet : couches applicatives, composants principaux et leurs interactions.]

Diagramme d'architecture

Insérer ici un diagramme d'architecture via `\includegraphics[width=]{architecture}.pdf`.

L'architecture repose sur les composants suivants :

- ▶ **Front-end** : interface utilisateur en HTML/CSS/JavaScript.
- ▶ **Back-end** : serveur applicatif exposant une API REST.
- ▶ **Base de données** : stockage persistant des données.
- ▶ **Services externes** : [API tierces, services mail, etc.]

2 Description des composants applicatifs

2.1 Composant 1 : [Nom]

[Décrire le premier composant majeur de l'application.]

- ▶ **Rôle** : [description du rôle]
- ▶ **Technologies** : [technologies utilisées]
- ▶ **Interfaces** : [points d'entrée/sortie]

2.2 Composant 2 : [Nom]

[Décrire le second composant majeur.]

3 Architecture back-end

3.1 Structure du projet

[Décrire l'organisation des fichiers et répertoires du projet back-end.]

```
1 projet/  
2     app/  
3         models.py          # Mod les de donn es  
4         views.py           # Logique m tier / contr leurs  
5         urls.py            # Routage des URL  
6         serializers.py     # S rialisation (API REST)  
7         tests.py           # Tests unitaires  
8     config/  
9         settings.py        # Configuration g n rale  
10        wsgi.py  
11        requirements.txt  
12        manage.py
```

Listing 1 – Structure du projet

3.2 Modèles de données

[Décrire les principaux modèles de données.]

```
1 class Utilisateur(AbstractUser):  
2     email = models.EmailField(unique=True)  
3     role = models.CharField(  
4         max_length=20,  
5         choices=[('user', 'Utilisateur'), ('admin', '  
6             Administrateur')],  
7         default='user'  
8     )  
9     date_inscription = models.DateTimeField(auto_now_add=True)  
10  
11     USERNAME_FIELD = 'email'  
12     REQUIRED_FIELDS = ['username']
```

Listing 2 – Exemple de modèle utilisateur

3.3 API REST

[Décrire les endpoints principaux de l'API.]

Méthode	Endpoint	Description
POST	/api/auth/login/	Connexion utilisateur
POST	/api/auth/register/	Inscription utilisateur
GET	/api/utilisateurs/	Liste des utilisateurs (admin)
GET	/api/[ressource]/	Lister les ressources
POST	/api/[ressource]/	Créer une ressource
PUT	/api/[ressource]/{id}/	Modifier une ressource
DELETE	/api/[ressource]/{id}/	Supprimer une ressource

4 Intégration des services externes

[Décrire comment les services externes sont intégrés dans l'application : API tierces, services d'e-mail, stockage, etc.]

4.1 Service de messagerie

[Décrire la configuration du service d'envoi d'e-mails.]

```

1 EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
2 EMAIL_HOST    = 'smtp.gmail.com'
3 EMAIL_PORT    = 587
4 EMAIL_USE_TLS = True
5 EMAIL_HOST_USER      = env('EMAIL_HOST_USER')
6 EMAIL_HOST_PASSWORD  = env('EMAIL_HOST_PASSWORD')
```

Listing 3 – Configuration e-mail (settings.py)

4.2 Intégration d'un service tiers

[Décrire l'intégration d'un service tiers spécifique au projet.]

5 Traitement des données

[Décrire comment les données sont traitées, stockées et sécurisées.]

5.1 Formats pris en charge

- Format 1 : [description]
- Format 2 : [description]

5.2 Pipeline de traitement

[Décrire le pipeline de traitement des données si applicable.]

1. **Étape 1** : Réception des données.
2. **Étape 2** : Validation et nettoyage.
3. **Étape 3** : Traitement métier.
4. **Étape 4** : Stockage en base de données.

6 Fonctionnalités additionnelles

6.1 Historique et journalisation

[Décrire les mécanismes de journalisation des actions.]

6.2 Accessibilité

[Décrire les mesures d'accessibilité mises en place.]

7 Sécurité de l'application

7.1 Authentification et gestion des sessions

- ▶ Authentification par token JWT / cookies de session.
- ▶ Expiration des sessions après [N] minutes d'inactivité.
- ▶ Hachage des mots de passe avec `bcrypt` / `PBKDF2`.

7.2 Contrôle d'accès

- ▶ Vérification des permissions à chaque requête.
- ▶ Séparation des rôles : utilisateur standard / administrateur.

7.3 Sécurité des données

- ▶ Protection contre les injections SQL via l'ORM.
- ▶ Protection CSRF sur les formulaires.
- ▶ Validation stricte des entrées utilisateur.
- ▶ Conformité RGPD : droit d'accès, modification et suppression des données.

7.4 Tests de sécurité

[Décrire les tests de sécurité réalisés.]

8 Conclusion

[Synthétiser les choix techniques principaux et les perspectives d'évolution de l'architecture.]

Ce document a présenté l'ensemble des choix techniques du projet [Nom]. L'architecture mise en place permet [résumé des avantages]. Les perspectives d'évolution incluent [amélioration 1] et [amélioration 2].